Java Development Tools

JDK

JDK (Java Development Kit) is the set of tools for building Java programs.
It includes the compiler, runtime, and libraries.

IntelliJ IDEA

IntelliJ IDEA is an IDE (integrated development environment) for writing and managing code.
It is especially popular for Java development.

JAVA BASICS

Anatomy of a Java Program

Example

```java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello!");
    }
}
```

keyword · class/type · method · string

Main parts

Class: HelloWorld contains the program.

Main method: main is where the program starts.

Statement: System.out.println("Hello!") tells Java what to do. Statements end with a semicolon.

Braces: { } mark the beginning and end of code blocks.

Naming conventions

Classes use PascalCase: HelloWorld.

Methods use camelCase and usually start with a verb: printMessage().

A public class name should match its filename: HelloWorld.java.

Packages

A package groups related classes and helps prevent naming conflicts.

Example: package com.example.app; | Folder: com/example/app

Create a Package in IntelliJ IDEA

Packages keep related classes organized and names distinct.

Create the package

1. In the Project panel, expand your project, then src.
2. Right-click src and choose New > Package.
3. Enter a lowercase name, such as

```
com.example.app
```

4. Move Main.java into the package. Choose Refactor if IntelliJ asks.

What IntelliJ adds to Main.java

```
package com.example.app;
```

File path: src/com/example/app/Main.java

The main method

Why the traditional entry point is static

NOTE:

`static` lets Java run **`main`** without creating an object first. Keep it in your program's **`main`** method.

How Java Code Gets Executed

Java code moves through compilation and execution.

1. Compilation: source code to bytecode



Java source code is saved in a **`.java`** file, such as **`Main.java`**. Compile it from the folder containing the file:

```
javac Main.java
```

`javac` is the Java compiler. It creates **`Main.class`**, which contains bytecode - instructions for the Java Virtual Machine. The compiler comes with the JDK, which provides tools for developing Java programs.

2. Execution: bytecode to a running program

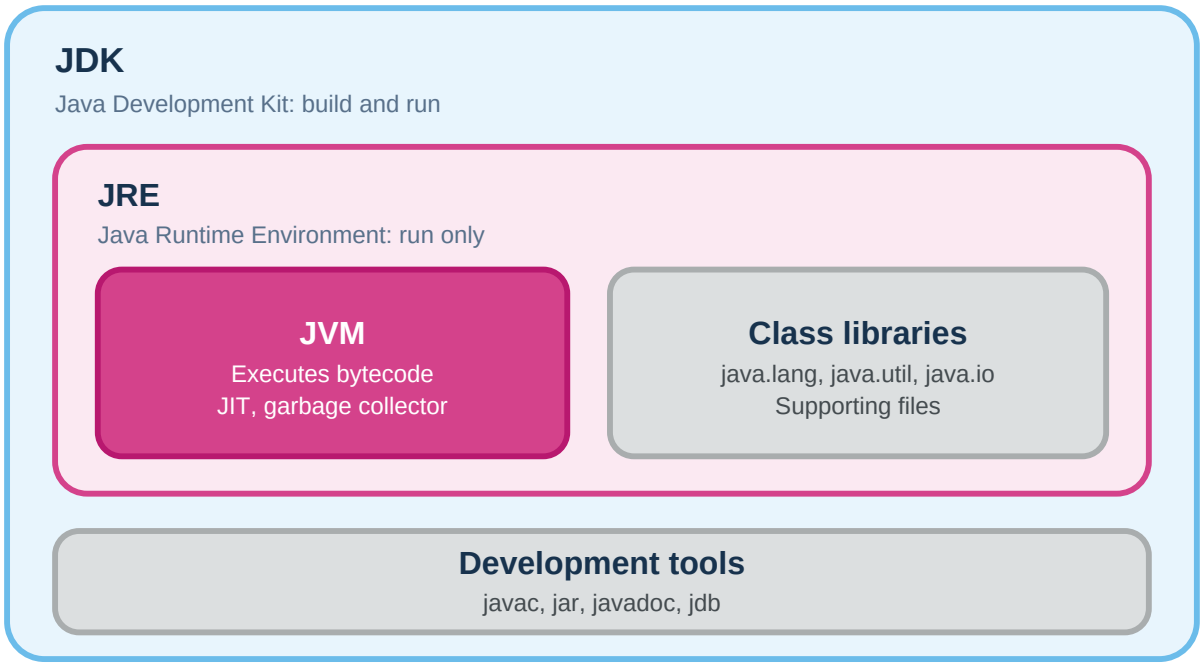


The **JVM (Java Virtual Machine)** loads and runs the bytecode in a `.class` file. It can translate frequently used parts into native machine instructions for the computer.

The **JRE (Java Runtime Environment)** provides the JVM and the Java libraries needed to run programs. In simple terms, the JVM runs the bytecode, and the JRE provides the runtime environment it needs.

Each operating system has a compatible JVM. That lets the same Java bytecode run on different systems without recompiling the source for each one.

3. JDK, JRE and JVM together



Because the JDK is built in layers, installing the outer one installs everything inside it.

- The JDK contains the JRE, since you need a way to run your code after you compile it.
- The JRE contains the JVM, since running Java code means executing bytecode, which only the JVM does.

So one JDK install gives you the compiler and tools (`javac`, `jar`, and so on), the class libraries, and the JVM. When you run `javac` and then `java`, the first uses the JDK's tools and the second launches the JVM from the bundled runtime.

Java Editions



Standard Edition (SE)

General-purpose Java for desktop and server applications.

Enterprise Edition (EE)

Adds APIs and services for large web and business applications.

Micro Edition (ME)

Designed for embedded and resource-limited devices, such as sensors.

Java Card

For secure applications on smart cards and other devices with very limited resources.

For most beginners: start with Java SE. Java EE is now known as Jakarta EE.

Variables

Variables store values that a program can use and update.

```
int age = 30;
System.out.println("Age: " + age);
```

Declare and use a variable

int is the type and stores whole numbers. age is the variable name; 30 is its starting value. The print line joins the label and value, so it displays Age: 30.

```
int temperature = 45;
temperature = 60;
System.out.println("Temperature: " + temperature);
```

Change its value

temperature starts at 45, then is updated to 60. Since it was already declared, the second line uses the name only. The print statement displays Temperature: 60.

```
int myAge = 26;
int herAge = myAge;
System.out.println("Her Age: " + herAge);
```

Copy a value

herAge receives the current value of myAge (26). Each int variable stores its own value, so changing one later does not automatically change the other.

Remember: declare a variable with its type once, then use its name to read or change it.

Primitive Types

Main.java demonstrates five of Java's eight built-in primitive types.

Primitive	Reference / non-primitive
Stores a simple value, such as a number or true/false.	Stores a reference to an object, such as a String or array.

```
byte age = 30;

long viewsCount = 3_123_456_789L;

float price = 10.99F;

char letter = 'A';

boolean isEligible = false;
```

byte small whole number

long large whole number; L marks long, underscores group digits

float decimal value; F marks float

char one character in single quotes

boolean true or false

All eight primitive types

Type	Size	Used for
byte	1 byte	Small whole numbers: -128 to 127
short	2 bytes	Whole numbers: -32,768 to 32,767
int	4 bytes	Whole numbers: -2,147,483,648 to 2,147,483,647
long	8 bytes	Very large whole numbers
float	4 bytes	Decimal values, about 6-7 significant digits
double	8 bytes	Decimal values, about 15-16 significant digits
char	2 bytes	One UTF-16 code unit
boolean	not specified	true or false; Java sets no fixed storage size

Reference (Non-Primitive) Types

Main.java uses Date; String is another common reference type.

Reference type

A reference variable stores a reference to an object. Date and String are examples.

Date example

```
import java.util.Date;


Date now = new Date();

System.out.println(now);
```

What happens

Date is a class in the `java.util` package, so the import lets this file use the short name **Date**. `new Date()` creates an object for the current date and time. **now** refers to that object, and printing it displays its date and time.

String example

```
String message = "Hello, Java!";

System.out.println(message);
```

What happens

String is also a reference type. The text in double quotes creates the string value, and **message** refers to it. Java makes `String` available automatically, so no import is needed.

Output: Hello, Java!

Important: Java creates a **String** object for a string literal, so `"Hello, Java!"` needs no `new`. **Date** has no literal syntax, so `new Date()` creates the object.

Null and Reference Variables

A reference can point to an object - or to no object at all.

null is a special value meaning 'no object'. Since String is a reference type, a String variable can hold null.

Three String values

```
String name = "Amina";
String missing = null;
String empty = "";

System.out.println(name);
System.out.println(empty.length());
```

A string literal creates a String value automatically. Writing null means the variable refers to no String object.

How they differ

- **`name = "Amina"`**
  Refers to a String object containing text.
- **`missing = null`**
  Refers to no object; it has no text or length.
- **`empty = ""`**
  Refers to a real String object with zero characters.

Important: null is not an empty String

Calling a method on null, such as missing.length(), causes a NullPointerException at runtime because there is no object to receive the call. Calling empty.length() is safe and returns 0.

Check before using a possibly null reference

```
if (name != null) {
    System.out.println(name.length());
} else {
    System.out.println("No name");
}
```

The condition confirms that name refers to an object before length() is called. Use this pattern when a value may be absent.

Remember

String is a reference type, so it can be null. A quoted string such as "Java" and an empty string "" are both actual String objects.

Primitives, Wrappers, and null

Primitive variables hold values; reference variables hold references to objects.

int and float are primitive types, so they cannot store null. Integer and Float are reference types, so they can.

Primitive types

```
int count = 0;
float price = 0.0f;

// Not allowed:
// int count = null;
```

A primitive stores its value directly. There is no object reference to set to null. Assign a meaningful value such as 0 when that fits your program.

Wrapper types

```
Integer optionalCount = null;
Float optionalPrice = null;

Integer score = 95;
```

Integer and Float are classes that wrap primitive numbers in objects. Use them when a number may be missing, such as an unanswered form field.

When does Java assign a default value?

Fields and array elements receive defaults: numeric primitives become 0 (or 0.0), boolean becomes false, and reference types become null. Local variables inside a method do not get an automatic default - assign them before reading them.

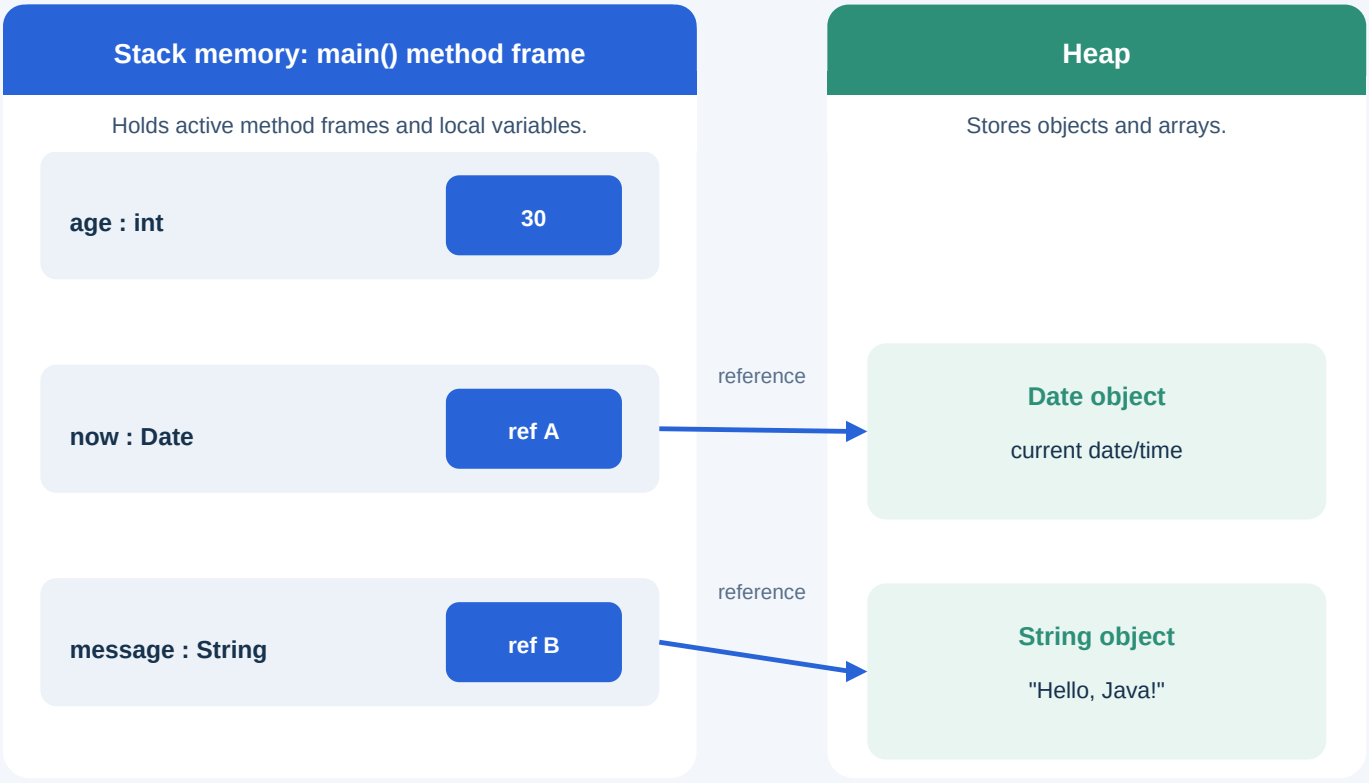
Choosing the right type

Boxing converts a primitive into its wrapper object; unboxing gets the primitive value back. Java can do these conversions automatically.

```
int count = 5;
Integer boxed = count;  // boxing: int -> Integer
int copied = boxed;    // unboxing: Integer -> int
Integer missing = null;
int unsafe = missing;  // NullPointerException
```

Where Primitive and Reference Variables Live

A simple memory model for local variables inside main().



Primitive local

```
int age = 30;
```

The local variable contains the value **30** directly in the current method frame.

Stack memory: Each thread's stack holds its active method frames and local variables.

Reference local

```
Date now = new Date();
String message = "Hello, Java!";
```

now and message hold references in the stack frame. Their Date and String objects are stored in the heap.

Heap memory: Shared memory where objects and arrays are created.

Remember: This is the JVM's conceptual model for local variables. Runtime optimizations can change their physical placement.

Working with Strings

Useful methods for checking and changing text

Important: String is immutable: these methods return a result without changing the original text.

The examples below all use the same String variable.

Example

```
String text = "  Java Programming  ";
System.out.println(text.length());
System.out.println(text.trim());
System.out.println(text.replace("Java", "Python"));
System.out.println(text.indexOf("Programming"));
System.out.println(text.endsWith(" "));
System.out.println(text.toUpperCase());
System.out.println(text.toLowerCase());
```

What each method does

Method	Purpose	Result for this example
`length()`	Returns the String's length.	`20`
`trim()`	Removes spaces from the beginning and end.	`"Java Programming"`
`replace("Java", "Python")`	Replaces matching text with other text.	`" Python Programming "`
`indexOf("Programming")`	Returns the first match's zero-based position; -1 if absent.	`7`
`endsWith(" ")`	Checks whether the text ends with the given suffix.	`true`
`toUpperCase()`	Returns the text in uppercase.	`" JAVA PROGRAMMING "`
`toLowerCase()`	Returns the text in lowercase.	`" java programming "`

Remember: trim() removes only outer spaces; indexOf() starts counting at 0. endsWith() is case-sensitive. The original text variable stays unchanged.

Escape Sequences

Use a backslash to include special characters in a String

Key idea: A backslash starts an escape sequence inside a Java string literal.

Write the sequence in your code; Java converts it when the program runs.

Examples from Main.java

```
String message = "Hello \"Salik\"";
System.out.println(message);

String directory = "c:\\Windows\\....";
System.out.println(directory);

String tab = "c:\tWindows\\....";
System.out.println(tab);

String newLine = "c:\nWindows\\....";
System.out.println(newLine);
```

What each sequence does

Sequence	Purpose	What the example prints
`\"`	Adds a double quote inside the text.	`Hello "Salik"`
`\\`	Adds one literal backslash.	`c:\Windows\....`
`\t`	Adds a tab character.	`c: [tab] Windows\....`
`\n`	Starts a new line.	`c: then a new line: Windows\....`

Windows paths: write each backslash as \\, because a single backslash begins an escape sequence.

Use "c:\\Windows" in Java; it prints c:\Windows.

Arrays

Store and work with several values of the same type

Array: A fixed-size collection of values, all of the same type.

An array variable refers to an array object; indexes start at 0.

Create and read

Import java.util.Arrays to print every value together.

```
import java.util.Arrays;
int[] numbers = {10, 20, 30};
System.out.println(numbers[0]); // 10
System.out.println(Arrays.toString(numbers));
```

Full-array output: [10, 20, 30].

Update and check size

Assign a new value or read the array's length.

```
numbers[1] = 25;
System.out.println(numbers[1]); // 25
System.out.println(numbers.length); // 3
```

Use .length without parentheses.

Loop through an array

Print each element in order.

```
for (int i = 0; i < numbers.length; i++) {
    System.out.println(numbers[i]);
}
```

Output

```
10
20
30
```

Create an array with a size

Choose the number of elements when creating it.

```
int[] scores = new int[3];
scores[0] = 90;
// {90, 0, 0}
```

Unassigned int elements start at 0.

Remember: Arrays have a fixed size and are reference types.

The last valid index is length - 1. An invalid index causes an error.

Multidimensional Arrays

Organize values in rows and columns

2D array: An array of arrays. Use [row][column] to select a value.

Row and column indexes both start at 0.

Create and access

Access an item by its row and column.

```
int[][] scores = {
    {80, 90, 85},
    {70, 75, 95}
};
System.out.println(scores[0][1]); // 90
```

scores[0][1] means row 0, column 1: 90.

Print the whole array

Use deepToString to show nested values.

```
import java.util.Arrays;
System.out.println(Arrays.deepToString(scores));
```

Output

```
[[80, 90, 85], [70, 75, 95]]
```

Arrays.toString() won't show nested contents.

Loop through every value

The outer loop visits rows; the inner loop visits each row's values.

```
for (int row = 0; row < scores.length; row++) {
    for (int col = 0; col < scores[row].length; col++) {
        System.out.println(scores[row][col]);
    }
}
```

Output

```
80
90
85
70
75
95
```

Remember: scores.length is the number of rows.

scores[row].length is the number of columns in that row.

Constants

A value you assign once and keep unchanged

In Main.java: PI is declared with final, so its value cannot be reassigned.
The program prints the constant's value: 3.14.

Example

```
final float PI = 3.14F;
System.out.println(PI);
```

Output

```
3.14
```

What final means

final lets you assign the variable once.
Trying to change it later causes a compile error:

```
PI = 3.14159F;
```

Because PI is final, Java won't allow this assignment.

The value 3.14 is an approximation of pi.

Naming and float values

Constant names are commonly written in uppercase, such as PI or MAX_USERS.

Decimal literals are double by default. The F suffix marks 3.14F as a float value.

Uppercase is a naming convention; final enforces the rule.

Remember: final controls whether the value can be reassigned.

The uppercase name is a convention that makes constants easy to recognize.

Arithmetic Operators

Calculate values and change a number by one

Operators: + add - subtract * multiply / divide % remainder

The code examples below show each operator in use.

Basic calculations

Use these operators to work with numbers.

```
int a = 10;
int b = 3;
System.out.println(a + b); // 13
System.out.println(a - b); // 7
System.out.println(a * b); // 30
System.out.println(a % b); // 1
```

% gives the remainder after division.

Integer and decimal division

With integers, division drops the fractional part.

```
int whole = 10 / 3; // 3
double decimal = 10.0 / 3;
System.out.println(whole); // 3
System.out.println(decimal); // 3.3333333333333335
```

10.0 is a double, so the result keeps a fractional part.

Increment and decrement

++ adds 1; -- subtracts 1.

```
int count = 5;
count++;
System.out.println(count); // 6
count--;
System.out.println(count); // 5
```

On its own, count++ and ++count both add one.

Order of operations

Multiplication happens before addition; parentheses go first.

```
int result = 2 + 3 * 4; // 14
int grouped = (2 + 3) * 4; // 20
```

Use parentheses when you want to control the order.

Remember: 10 / 3 is integer division and gives 3; 10.0 / 3 gives a decimal result.

Use at least one double value when you want the fractional part of a division.

Casting and Type Conversion

Convert numeric values, or parse text into a number

Main.java: shows automatic conversion, an explicit cast, and parsing text.

Parsing a String to a number is different from casting a numeric value.

Automatic conversion

Java promotes numeric values when an expression needs a wider type.

```
short x = 1;
int y = x + 2;
System.out.println(y); // 3
```

The result is an int, so it fits in y.

Automatic conversion to double

The int value 2 is converted to double during the calculation.

```
double a = 1.1;
double b = a + 2;
System.out.println(b); // 3.1
```

No cast is needed for this widening conversion.

Explicit cast to int

Write the target type in parentheses before the value.

```
double q = 1.1;
int w = (int) q + 2;
System.out.println(w); // 3
```

(int) q drops the .1 first; casting truncates, it does not round.

Parse text into a number

Use Integer.parseInt when the value is stored as text.

```
String s = "1";
int r = Integer.parseInt(s) + 2;
System.out.println(r); // 3
```

This is parsing, not casting: a String is not a numeric type.

Remember: Widening is often automatic; narrowing needs an explicit cast.

Casting can lose information: (int) 1.9 becomes 1, not 2.

The Math Class

Useful built-in methods for calculations and random numbers

Math provides ready-made static methods.
Call them with Math.methodName(...); no import or Math object is needed.

Round, ceil, and floor

These methods change a decimal value in different ways.

```
Math.round(1.1F);
(int) Math.ceil(1.1F);
(int) Math.floor(1.1F);
```

Output 1; 2; 1

ceil rounds upward; floor rounds downward. Both return double, so the cast makes an int.

Smallest and largest

Compare two values with min and max.

```
Math.min(1, 2);
Math.max(1, 2);
```

Output 1; 2

min returns the smaller value; max returns the larger one.

A random decimal

Math.random() gives a new pseudo-random decimal each time.

```
double result = Math.random();
System.out.println(result);
```

Output For example: 0.583...

The value is at least 0.0 and always less than 1.0.

A random whole number

Multiply by 100, then cast to int to remove the decimal part.

```
int randomInt =
    (int) (Math.random() * 100);
System.out.println(randomInt);
```

Output For example: 42

randomInt can be any whole number from 0 through 99.

Remember: Math methods are static, so call them using the class name: Math.methodName(...).

The random integer example uses randomInt instead of the placeholder name x.

Number Formatting

Format numbers as currency or percentages

What is NumberFormat?

NumberFormat is a Java class that turns numbers into readable, formatted text.

It provides formatter objects for styles such as currency and percentages.

Currency

NumberFormat is the variable type. The factory method creates a formatter for money.

```
NumberFormat currency =
    NumberFormat.getCurrencyInstance();
String currencyResult =
    currency.format(1234567.891);
System.out.println(currencyResult);
```

Example output

Example: $1,234,567.89 (varies by locale).

currency is a formatter object. Its format() method returns a String with currency symbols and separators for the active locale.

Percentage

This factory method creates a formatter for percentage values.

```
NumberFormat percentage =
    NumberFormat.getPercentInstance();
String percentageResult =
    percentage.format(0.1);
System.out.println(percentageResult);
```

Example output

Example: 10% (format may vary by locale).

percentage is a formatter object. format() scales 0.1 by 100 and adds the locale's percent symbol.

Important: Why no new keyword?

NumberFormat is an abstract class, so you cannot create it directly with new.

Instead, getCurrencyInstance() creates and returns a suitable formatter for your locale:

```
NumberFormat currency =   NumberFormat.getCurrencyInstance();
```

An object is still created; the factory method creates it and gives it back to you.

JAVA BASICS

Reading Input

Use Scanner to read values typed by the user

First, import Scanner and create it inside main():

```
import java.util.Scanner;   Scanner scanner = new Scanner(System.in);
```

nextLine()

Reads the rest of the current line, including spaces.

```
String fullName = scanner.nextLine();
```

Input: Ada Lovelace -> fullName stores both words.

next()

Reads one token, stopping at whitespace.

```
String firstWord = scanner.next();
```

Input: Ada Lovelace -> firstWord is "Ada".

nextByte()

Reads a whole number that fits in byte range (-128 to 127).

```
byte age = scanner.nextByte();
```

Example input: 25 -> age stores 25.

nextFloat()

Reads a decimal number as a float.

```
float price = scanner.nextFloat();
```

Example input: 12.5 -> price stores 12.5f.

nextInt()

Reads a whole number as an int.

```
int quantity = scanner.nextInt();
```

Example input: 7 -> quantity stores 7.

There is no nextString()

Scanner has no nextString() method. Use one of these:

```
String word = scanner.next();
String line = scanner.nextLine();
```

Use next() for one word, or nextLine() for a full line.

Why can nextLine() return an empty line after a number?

nextInt() reads the number but leaves the Enter key in the input.

The nextLine() may read that Enter and return an empty line.

Clear the leftover Enter, then call nextLine() again to read the text:

```
int age = scanner.nextInt();
scanner.nextLine();  // consume Enter
String name = scanner.nextLine();
```

JAVA BASICS

Comparison Operators

Compare values to produce a true or false result

These operators compare two values.

In Java, every comparison evaluates to either true or false.

Operator	Meaning	Example (age = 20)
==	equal to	`age == 20  ->  true`
!=	not equal to	`age != 18  ->  true`
>	greater than	`age > 18  ->  true`
<	less than	`age < 18  ->  false`
>=	greater than or equal to	`age >= 20  ->  true`
<=	less than or equal to	`age <= 19  ->  false`

Example

Run these comparisons and check the printed results:

```
int age = 20;
System.out.println(age == 20);  // true
System.out.println(age != 18);  // true
System.out.println(age > 18);   // true
System.out.println(age < 18);   // false
System.out.println(age >= 20); // true
System.out.println(age <= 19); // false
```

Important for Strings

Use .equals() to compare String text, for example: name1.equals(name2).

The == operator checks whether two references point to the same object.

JAVA BASICS

Logical Operators

Combine or reverse conditions in your program

Logical operators work with true/false conditions.

They help a program make decisions using more than one condition.

Operator	Meaning	Real-world scenario
&&	AND: both conditions must be true.	Rent a car if age >= 21 AND you have a valid license.
||	OR: at least one condition must be true.	A day off if it is a weekend OR a public holiday.
!	NOT: reverses true and false.	Go for a walk if it is NOT raining.

Java example

The expressions below model the same everyday decisions:

```
int age = 22;
boolean hasLicense = true;
boolean canRent = age >= 21 && hasLicense;
boolean isWeekend = true;
boolean isHoliday = false;
boolean dayOff = isWeekend || isHoliday;
boolean isRaining = false;
boolean canGoForWalk = !isRaining;
System.out.println(canRent);
System.out.println(dayOff);
System.out.println(canGoForWalk);
```

Output `true` `true` `true`

The car can be rented, it is a day off, and a walk is possible.

JAVA BASICS

Making Decisions with if

Choose which code runs based on a condition

A condition evaluates to true or false.

Java uses if, else, and else if to choose an action based on that result.

Statement	How it chooses	Everyday scenario
`if`	Runs the code when the condition is true.	If your balance covers the cost, approve the purchase.
`if / else`	Chooses one of two paths.	If it rains, take an umbrella; otherwise, leave it at home.
`else if`	Checks several conditions in order.	Choose a ticket price based on the customer's age.

Write else if as two words. Conditions are checked from top to bottom; the first true branch runs.

Example: set a ticket price by age

Each else if checks the next age range only if earlier conditions were false.

```
int age = 10;
int ticketPrice;
if (age < 5) { ticketPrice = 0; }
else if (age < 13) { ticketPrice = 5; }
else if (age < 65) { ticketPrice = 10; }
else { ticketPrice = 7; }
System.out.println("Ticket price: $" + ticketPrice);
```

Output: Ticket price: $5

Simplifying an if/else that only sets true or false

LONGER VERSION

```
boolean adult;
if (age >= 18) adult = true;
else adult = false;
```

SIMPLER VERSION

```
boolean adult = age >= 18;
```

age >= 18 already gives true or false, so assign its result directly.

JAVA BASICS

Ternary Operator

A compact way to choose one of two values

The ternary operator checks a condition and selects one of two values.

```
condition ? valueIfTrue : valueIfFalse
```

CONDITION

A true/false test

? TRUE VALUE

Used when condition is true

: FALSE VALUE

Used when condition is false

Real-world example: show a pass or retry message

A score of 50 or higher passes; otherwise, the student should try again.

```
int score = 72;

String result = score >= 50 ? "Pass" : "Try again";

System.out.println(result);
```

Output `Pass`

Equivalent if/else

The ternary version is shorter when each branch just supplies a value.

IF/ELSE

```
String result;
if (score >= 50) result = "Pass";
else result = "Try again";
```

TERNARY

```
String result = score >= 50
    ? "Pass" : "Try again";
```

Use if/else for longer actions or several choices; it is easier to read.

JAVA BASICS

Switch Statements

Choose an action that matches a value

switch checks one value against several cases.

Use it for choices such as menu options or days of the week. It supports int and String values.

Example 1: choose a café drink with an int

```
int choice = 2;
switch (choice) {
    case 1:
        System.out.println("Coffee");
        break;
    case 2:
        System.out.println("Tea");
        break;
    default:
        System.out.println("Invalid choice");
        break;
}
```

Output: Tea

Example 2: respond to a day name with a String

```
String day = "SATURDAY";
switch (day) {
    case "MONDAY":
        System.out.println("Workday");
        break;
    case "SATURDAY":
        System.out.println("Weekend");
        break;
    case "SUNDAY":
        System.out.println("Weekend");
        break;
    default:
        System.out.println("Unknown day");
        break;
}
```

Output: Weekend

`case`

A value to match

`break`

Exits the switch

`default`

Runs when none match

In this switch style, break prevents running into the next case; default is optional.

JAVA EXERCISE

FizzBuzz

Practice using if, else if, modulo, and user input

The exercise

Read an integer and print FizzBuzz if divisible by 3 and 5, Fizz if divisible by 3, Buzz if divisible by 5, or the number itself otherwise.

Try these values

Input 9	Input 10	Input 15	Input 7
Fizz	**Buzz**	**FizzBuzz**	**7**

One possible solution

Save as FizzBuzz.java. The combined condition must be checked first.

```java
import java.util.Scanner;
public class FizzBuzz {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int number = scanner.nextInt();
        if (number % 3 == 0 && number % 5 == 0) {
            System.out.println("FizzBuzz");
        } else if (number % 3 == 0) {
            System.out.println("Fizz");
        } else if (number % 5 == 0) {
            System.out.println("Buzz");
        } else {
            System.out.println(number);
        }
    }
}
```

Example: input 15 prints FizzBuzz

Why check both first?

15 is divisible by both 3 and 5. Checking it first ensures the program prints FizzBuzz, instead of stopping at the first matching condition. The % operator gives a remainder; a remainder of 0 means the number is divisible evenly.

JAVA BASICS

Java Loops

Repeat code and choose the right loop for the task

Loops repeat instructions while a task needs to continue.

Use for when repetitions are known; while and do-while when repetition depends on a condition.

`for`

Use when you know how many times to repeat.

`while`

Check first; repeat while condition is true.

`do-while`

Run once, then check whether to repeat.

For the input examples, import java.util.Scanner and create:

```
Scanner scanner = new Scanner(System.in);
```

for loop example: average three quiz scores

The number of scores is fixed, so a for loop is a natural fit.

```
int total = 0;
for (int i = 1; i <= 3; i++) {
    System.out.print("Score " + i + ": ");
    total += scanner.nextInt();
}
double average = total / 3.0;
System.out.println("Average: " + average);
```

Sample input: 80, 90, 100 **Output: Average: 90.0**

How the for loop works

int i = 1 starts the counter. i <= 3 keeps the loop going through score 3.

i++ adds 1 after each pass, so the loop runs exactly three times.

The braces hold the instructions repeated on every pass.

JAVA BASICS

while and do-while

Use user input to decide when a loop should stop

These examples use the same Scanner setup from the previous page.

Read a value with scanner.nextInt(), then use it to control the loop.

while loop: total expenses until the user enters 0

Read the first expense before the loop; while checks the condition before each pass.

```
int total = 0;
        System.out.print("Expense (0 to finish): ");
int expense = scanner.nextInt();
while (expense != 0 ) {
    total += expense;
    System.out.print("Expense (0 to finish): ");
    expense = scanner.nextInt();
}
System.out.println("Total: " + total);
```

Input: 12, 8, 0 **Output: Total: 20**

do-while loop: ask for a valid service rating

The prompt runs at least once; the loop repeats while the rating is outside 1-5.

```
int rating;
do {
    System.out.print("Rate service 1-5: ");
    rating = scanner.nextInt();
    if (rating < 1 || rating > 5 ) {
        System.out.println("Please enter 1-5.");
    }
} while (rating < 1 || rating > 5 );
System.out.println("Thank you!");
```

Sample input: 8, then 5 **Output: Please enter 1-5. then Thank you!**

Remember the difference

while tests before the body, so it can run zero times. do-while tests after the body, so it always runs once.

A do-while statement ends with a semicolon after while (condition); as shown above.

JAVA BASICS

break and continue

Control what happens inside a loop

Both keywords change the loop's normal flow.

break exits the loop; continue skips to the next pass.

break: stop after the correct PIN is entered

A user gets up to three attempts. A correct PIN ends the retry loop immediately.

```
for (int attempt = 1; attempt <= 3; attempt++) {
    System.out.print("Enter PIN: ");
    int pin = scanner.nextInt();
    if (pin == 1234) {
        System.out.println("Access granted");
        break;
    }
    System.out.println("Incorrect PIN");
}
```

Example: first PIN is wrong; second is correct -> loop stops after access is granted.

continue: skip an invalid meter reading

Keep checking four readings, but do not record a negative value.

```
for (int i = 1; i <= 4; i++) {
    System.out.print("Enter reading: ");
    int reading = scanner.nextInt();
    if (reading < 0) {
        System.out.println("Invalid, skipped");
        continue;
    }
    System.out.println("Recorded: " + reading);
}
```

Input: 12, -1, 18, 20 -> records 12, skips -1, then records 18 and 20.

Remember

break exits the current loop completely, so no later passes happen.

continue skips the rest of the current pass; the loop then checks its condition and continues.

Both examples assume Scanner is imported and scanner is initialized inside main().

These keywords affect the nearest enclosing loop.

JAVA BASICS

for-each Loop

Visit every item in an array or collection

Real-world scenario: print each task on a to-do list.

A for-each loop takes each String from the array in order, so you do not need to manage an index like i yourself.

Example: display a list of study tasks

Each pass stores the next task in the variable task, then prints it.

```
String[] tasks = {"Study Java", "Practice coding", "Review notes"};

for (String task : tasks) {

    System.out.println("Task: " + task);

}
```

Output

```
Task: Study Java
Task: Practice coding
Task: Review notes
```

Read the loop

`String task`	The type and temporary variable that hold one item.
`: tasks`	Read as 'from tasks'; task receives each item.
`{ ... }`	The statements inside run once for every item.

For arrays, for-each visits items from first to last.
It gives no index or built-in way to go backward; use a regular for loop for either.

JAVA BASICS

Methods in Java

Reusable blocks of code that perform a task

A method is a named, reusable block of code that performs a task.

Real-world example: calculate and display the total for a purchase.

Example: calculate a purchase total

Calculate the cost of three notebooks, then display the result.

```
public class Main {
    public static void main(String[] args) {
        double total = calculateTotal(12.50, 3);
        printTotal("Notebook", total);
    }

    static double calculateTotal(double price, int quantity) {
        return price * quantity;
    }

    static void printTotal(String item, double total) {
        System.out.println(item + " total: $" + total);
    }
}
```

Output

```
Notebook total: $37.5
```

Parts of a method

`calculateTotal(12.50, 3)`	Calls the method; 12.50 and 3 are arguments.
`price, quantity`	Parameters: named inputs the method receives.
`double / return`	The method returns a number to the caller.
`void printTotal(...)`	Performs an action and returns no value.

Note: These methods are static so main can call them directly.

Java also has instance methods, which are called through objects.

JAVA BASICS

Refactoring

Improve code structure while keeping behavior the same

Definition: changing the structure of code without changing its behavior.

The program should produce the same result before and after refactoring.

Example: move a calculation into a method

The calculation and output stay the same; only the code organization changes.

Before - calculation inside main()

```
double total = 15.0 * 2;
System.out.println("Total: $" + total);

Output: Total: $30.0
```

After - extract the calculation into a method

In main()

```
double total = calculateTotal(15.0, 2);
System.out.println("Total: $" + total);
```

Define this method in the class

```
static double calculateTotal(double price, int quantity) {
return price * quantity;
}

Output: Total: $30.0
```

What changed?

The multiplication now lives in a named method, making the code easier to read and reuse.

The same input still produces the same output.

JAVA BASICS

Types of Errors

When the error is found determines when the program stops

Compile-time Errors

The compiler finds these before the program can run.

Example: missing semicolon

```
int age = 20

Missing ;
```

Java reports a compile error, so the program will not start.

Syntax errors are one kind of compile-time error. Type errors can also stop compilation.

Run-time Errors

These happen after the program compiles and begins running.

Example: invalid array index

```
int[] scores = {90, 85};

System.out.println(scores[3]);
```

The array has indexes 0 and 1; index 3 does not exist.

The code compiles, then throws `ArrayIndexOutOfBoundsException` when that line runs.

Quick difference: compile-time errors block the program from starting; run-time errors occur while it is executing.

JAVA BASICS

Debugging in IntelliJ IDEA

Pause the program and inspect values to find a bug

Debugging: Finding and removing errors in a program.

IntelliJ's debugger helps you pause execution and inspect program values.

Example: find a calculation bug

The program prints 13, but the expected total is 30.

Buggy code

```
int price = 10;

int quantity = 3;

int total = price + quantity;

System.out.println(total);
```

```
Output: 13              Expected: 30
```

Debug it in IntelliJ IDEA

1. Click the gutter beside the total line to set a breakpoint.
2. Click Debug (the bug icon). Execution pauses before that line runs.
3. Choose Step Over once. Then check total in the Variables pane.
4. You will see 13 instead of 30; the line adds the numbers.
5. Change + to *, run again, and confirm the output is 30.

A breakpoint pauses at a chosen line; Step Over runs that line and moves on.

JAR Files in Java

Package compiled code and launch a console application

What is an artifact?

An artifact is the finished item created from a project's files and resources, ready to be used or shared. A JAR is a common Java artifact: a ZIP-based archive that can bundle compiled `.class` files and other resources.

What can a JAR contain?

```
app.jar
META-INF/MANIFEST.MF
com/example/Main.class
```

→

One archive file

holds class files, manifest, and resources such as images or configuration files.

Make a JAR runnable

A runnable JAR needs a manifest that tells Java which class to start. The manifest is a text file at `META-INF/MANIFEST.MF` inside the archive.

MANIFEST.MF

```
Manifest-Version: 1.0
Main-Class: com.example.Main
```

What does Main-Class mean?

`com.example.Main` is the fully qualified class name: package `com.example` plus class `Main`. Do not add `.java` or `.class`. That class must contain the `public static void main(String[] args)` entry point.

Run the application

```
java -jar app.jar
```

Java reads the `Main-Class` entry, starts that class, and runs its `main` method.

Dependencies

A regular JAR does **not automatically include external libraries**. Those libraries must be provided separately on the classpath, or packaged together in an executable ("fat") JAR.

JAVA BASICS

Package a Java App in IntelliJ IDEA

Create, build, and test a runnable JAR

Before you start: Run your program successfully in IntelliJ IDEA. Make sure its main class contains `public static void main(String[] args)`.

1 Open the artifact settings

Go to **File > Project Structure**, then choose **Artifacts**. Shortcut: `Ctrl+Alt+Shift+S` on Windows/Linux.

2 Create a JAR artifact

Click **+**, choose **JAR**, then **From modules with dependencies**.

3 Choose the module and main class

Select your project module. Beside **Main Class**, click Browse and choose the class with your `main` method. This sets the JAR launch point.

4 Choose how to include libraries

If your app uses external libraries and you want one self-contained JAR, choose **Extract to the target JAR**. Otherwise, libraries may need to be supplied separately.

5 Save and build the JAR

Click **OK**, then **Apply** and **OK**. Choose **Build > Build Artifacts**, select your app's `jar` artifact, then click **Build**.

6 Find and run the JAR

IntelliJ IDEA usually puts it under `out/artifacts/`. Open a terminal in the folder containing the JAR and run the command below.

```
java -jar YourApp.jar
```

If it does not start

Check that the selected main class is correct, rebuild the artifact, and confirm you are using a compatible Java runtime.

Programming Paradigms

Different ways to organize and build a program

A **programming paradigm** is a general way of structuring code. One application can combine several paradigms.

Procedural

Runs instructions in an order, often grouped into methods.

REAL-WORLD EXAMPLE

ATM withdrawal: verify the PIN, check the balance, deduct the amount, then dispense cash.

Event-driven

Runs code in response to events, such as clicks, messages, or input.

REAL-WORLD EXAMPLE

Online cart: update the total when a shopper changes an item quantity.

Functional

Uses functions to transform values while limiting changes to shared data.

REAL-WORLD EXAMPLE

Checkout: apply a discount function to each item price, then add the results.

Logic

Defines facts and rules, then asks queries that the system can solve.

REAL-WORLD EXAMPLE

Eligibility checker: find applicants whose income and credit score meet the rules.

Object-oriented

Models a program as objects that combine data with related actions.

REAL-WORLD EXAMPLE

Bank account: an object stores its balance and provides deposit and withdraw actions.

Aspect-oriented

Keeps shared tasks, such as logging or security, separate from the main program logic.

REAL-WORLD EXAMPLE

Banking app: apply the same security check and logging to many service methods.

Remember: These styles can be combined. Java programs often mix object-oriented, procedural, functional, and event-driven approaches; logic programming and aspect-oriented programming use specialized languages or frameworks.

Problem Solving

Choosing a solution that fits the problem

DEFINITION

Problem solving is the process of **defining a problem**, identifying and comparing possible solutions, and choosing the one that best solves it within the given **context and constraints**.

The process

1. Define the problem
2. Identify and compare possible solutions
3. Choose the solution that fits the context and constraints

Object-Oriented Programming

Build programs using objects that combine data and behavior

Object-oriented programming (OOP) organizes code around objects. Each object combines its own **data** with the **behavior** that works on that data.

Class and object

Class

A blueprint

BankAccount describes what an account stores and can do.

Object

A specific instance

Amina's account and Bilal's account are two separate objects.

What does an object contain?

Data (state)

For an account: its owner and current balance.

Behavior (methods)

Actions such as deposit() and withdraw().

Benefits of OOP

Reduced complexity

Split a large program into smaller objects.

Easier maintenance

Update an object's code in one place.

Code reuse

Create many objects from the same class.

Faster development

Build with reusable, focused components.

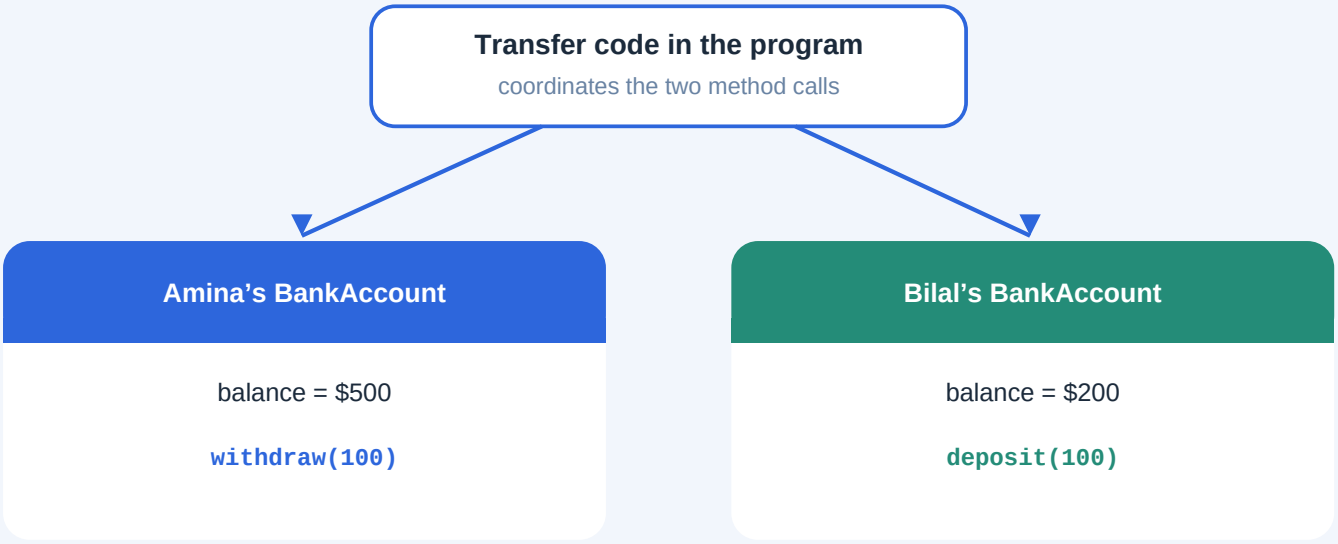
Objects created from the same class have the same structure, but each object keeps its own data.

Objects Working Together

A simple bank transfer using two account objects

Amina transfers **$100** to Bilal. The program asks one account object to withdraw the money and the other to deposit it.

How the objects interact



After the transfer: Amina has $400; Bilal has $300.

Java example

```
class BankAccount {
    int balance;
    BankAccount(int balance) { this.balance = balance; }
    void deposit(int amount) { balance += amount; }
    void withdraw(int amount) { balance -= amount; }
}

class Main {
    public static void main(String[] args) {
        BankAccount amina = new BankAccount(500);
        BankAccount bilal = new BankAccount(200);
        amina.withdraw(100);
        bilal.deposit(100);
        // balances are now 400 and 300
    }
}
```

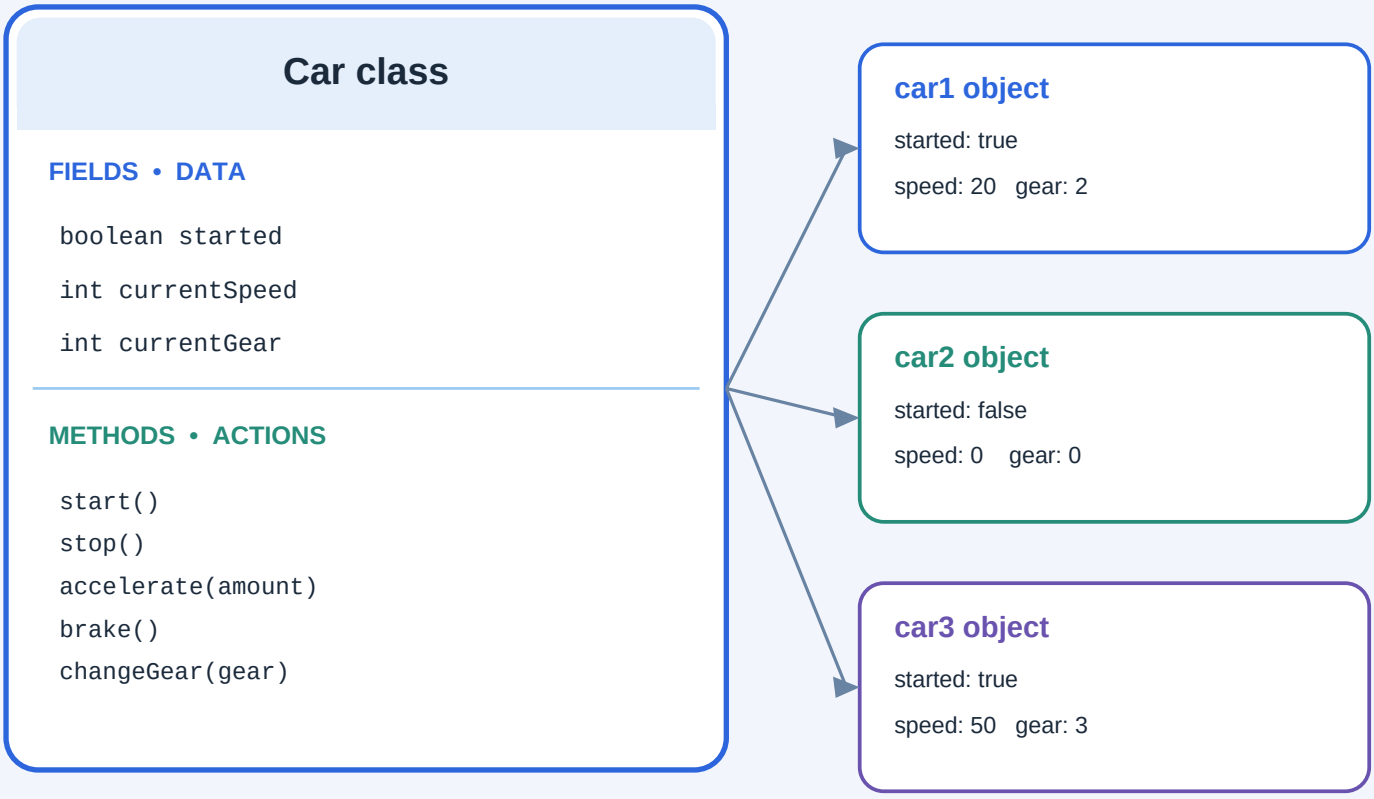
Each object keeps its own balance; method calls change the object they are called on.

Car Class and Its Objects

One class can be used to create many separate car objects

A **class** describes the data and actions for a kind of thing. An **object** is one specific instance created from that class.

The Car class is the blueprint



All objects use the same fields and methods, but each keeps its own field values.

Create three objects from the class

```
Car car1 = new Car();
Car car2 = new Car();
Car car3 = new Car();
```

Each `new Car()` creates a different object. Changing car1 does not change car2 or car3.

Car Objects in Java

Fields store each car’s state; methods change it

This example creates three cars, changes their state using methods, and prints each car’s values.

MAIN.JAVA

```
class Car {
    private boolean started;
    private int currentSpeed;
    private int currentGear;

    void start() { started = true; }
    void stop() {
        started = false;
        currentSpeed = 0;
        currentGear = 0;
    }
    void accelerate(int amount) {
        if (started) currentSpeed += amount;
    }
    void brake() {
        currentSpeed = Math.max(0, currentSpeed - 10);
    }
    void changeGear(int gear) { currentGear = gear; }
    void showStatus() {
        System.out.println("Started: " + started
            + ", speed: " + currentSpeed
            + ", gear: " + currentGear);
    }
}

public class Main {
    public static void main(String[] args) {
        Car car1 = new Car();
        Car car2 = new Car();
        Car car3 = new Car();
        car1.start();
        car1.accelerate(30);
        car1.changeGear(2);
        car1.brake();
        car3.start();
        car3.accelerate(50);
        car3.changeGear(3);
        car1.showStatus();
        car2.showStatus();
        car3.showStatus();
        car3.stop();
        car3.showStatus();
    }
}
```

What happens?

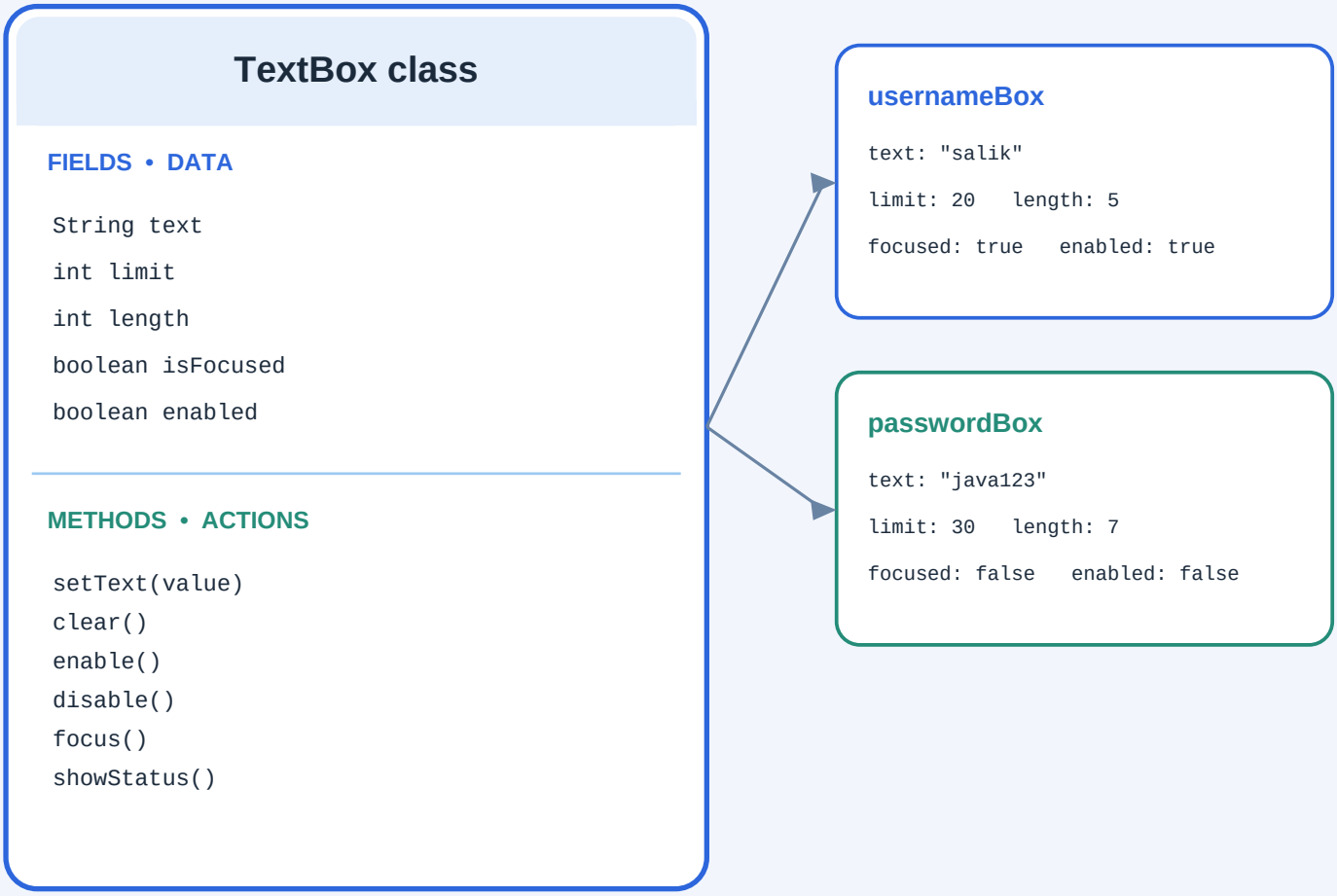
`car1.brake()` reduces car1’s speed by 10. car2 is untouched. `car3.stop()` resets its state. The three objects keep separate values.

TextBox Class and Objects

A form can create several TextBox objects from one class

A **TextBox** object represents an input field. It stores text and related state, and its methods change that state.

One class, two input fields



Each TextBox object has its own text, limit, length, and focus/enabled state.

Create two objects for a login form

```
TextBox usernameBox = new TextBox(20);

TextBox passwordBox = new TextBox(30);
```

The numbers set each field’s character limit.

TextBox Objects in Java

See how methods update each object’s own fields

A login form uses separate username and password TextBox objects. Text is accepted only when the field is enabled and the character limit is not exceeded.

MAIN.JAVA

```
class TextBox {
    private String text = "";
    private int limit;
    private int length;
    private boolean isFocused;
    private boolean enabled = true;

    TextBox(int limit) { this.limit = limit; }
    void setText(String value) {
        if (enabled && value.length() <= limit) {
            text = value;
            length = text.length();
        }
    }
    void clear() { text = ""; length = 0; }
    void enable() { enabled = true; }
    void disable() { enabled = false; isFocused = false; }
    void focus() { if (enabled) isFocused = true; }
    void showStatus() {
        System.out.println("text=" + text
            + ", length=" + length + ", limit=" + limit
            + ", focused=" + isFocused + ", enabled=" + enabled);
    }
}

public class Main {
    public static void main(String[] args) {
        TextBox usernameBox = new TextBox(20);
        TextBox passwordBox = new TextBox(30);
        usernameBox.setText("oldname");
        usernameBox.clear();
        usernameBox.focus();
        usernameBox.setText("salik");
        passwordBox.setText("java123");
        passwordBox.disable();
        passwordBox.setText("ignored"); // ignored while disabled
        usernameBox.showStatus();
        passwordBox.showStatus();
        passwordBox.enable();
    }
}
```

What the methods do

setText() checks enabled and limit, then updates text and length. clear() resets text and length. focus() sets isFocused. disable() blocks input; enable() allows it again.

username: text=salik | length=5 | limit=20 | focused=true | enabled=true

password: text=java123 | length=7 | limit=30 | focused=false | enabled=false

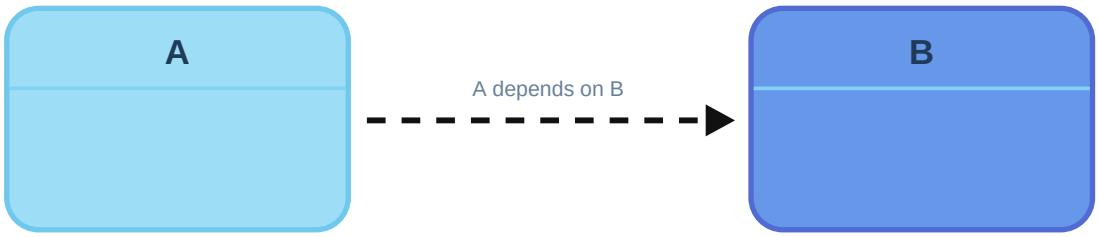
Coupling

How strongly classes depend on one another

DEFINITION

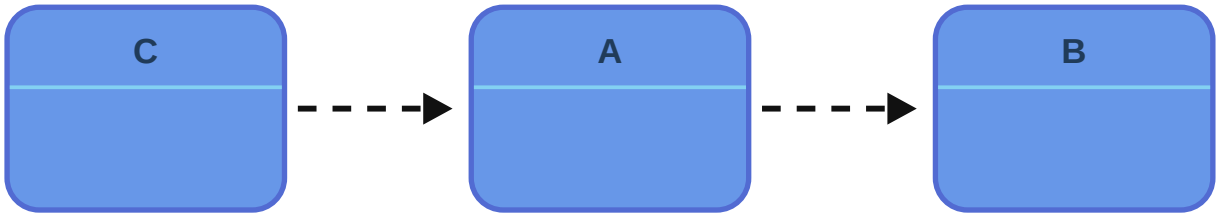
Coupling describes how much one class depends on another. Lower coupling usually makes classes easier to change independently.

Direct dependency: A uses B



The arrow points from the class that uses another class to the class it depends on. If A relies on B's public methods, changes to that interface may affect A.

Dependency chain: C uses A, and A uses B



A change to B's interface may affect A, and changes in A may then affect C. More dependencies can make a program harder to update.

Keep coupling low

Let classes use only the public methods they need. Keep helper methods private when other classes should not depend on them.

Coupling in Main and Browser

Main uses Browser through its public navigate() method

Main needs to know how to call navigate(). It does not need to know how Browser finds an IP address or sends a request.

Main.java

```
package com.salikhussain;

public class Main {
    public static void main(String[] args) {
        Browser browser = new Browser();
        browser.navigate("https://www.google.com/");
    }
}
```

Browser.java

```
package com.salikhussain;

public class Browser {
    public void navigate(String address) {
        String ip = findIpAddress(address, true);
        String html = sendHttpRequest(ip);
        System.out.println(html);
    }

    private String sendHttpRequest(String ip) {
        return "<html>Google...</html>";
    }

    private String findIpAddress(
            String address, boolean cache) {
        return "127.0.0.1";
    }
}
```

Why make findIpAddress() private?

If findIpAddress() were public and other classes called it, changing its signature to add cache would require those callers to update.

```
// Before: browser.findIpAddress(address);
// After:  browser.findIpAddress(address, true);
```

Because it is private, only call sites inside Browser need updating (here, navigate()). Main still calls navigate() unchanged. The cache parameter illustrates the signature change; this sample does not use its value yet.

Abstraction

A simple way to use something without learning every internal detail

DEFINITION

Abstraction reduces complexity by hiding unnecessary details and showing only what you need to use.

A car is a familiar example

You operate the car through a few controls; the car handles the complex work inside.

What the driver uses

Simple controls to operate the car

- Steering wheel
- Accelerator pedal
- Brake pedal

operate →

What happens inside

Details the driver does not need to manage

- Engine operation
- Transmission
- Braking mechanisms

How this reduces complexity

A driver can steer, accelerate, and stop the car without knowing how its engine and other systems work internally. The controls show the driver what they need to use and hide unnecessary implementation details.

In short Abstraction gives you a simple interface to use while the complex details stay behind the scenes.

Abstraction in Code: Car and Engine

Main uses one simple Car method; the objects handle the internal work

Main calls car.start(). Car uses Engine internally to perform the steps needed to start.

THREE CLASSES WORKING TOGETHER

```
package com.salikhussain;

class Engine {
    public void start() {
        checkFuel();
        ignite();
        System.out.println("Engine started");
    }

    private void checkFuel() {
        System.out.println("Fuel checked");
    }

    private void ignite() {
        System.out.println("Engine ignited");
    }
}

class Car {
    private Engine engine = new Engine();

    public void start() {
        engine.start();
        System.out.println("Car ready");
    }
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        car.start();
    }
}
```

Follow the call

Main `car.start()` → **Car** `engine.start()` → **Engine** `checkFuel(), ignite()`

Where is the abstraction?

Main only needs to know car.start(). It does not need to call checkFuel() or ignite(), or manage how Engine starts. Car offers a simple way to use the feature while Engine handles the hidden steps.

Encapsulation

Keep an object's data and related methods together in a class

DEFINITION

Encapsulation means bundling an object's data and the methods that work on it in one unit, such as a class, while keeping the data private and controlling access to it through methods.

Data

The information an object stores, such as a TextBox's text.

Methods

Actions that work on that data, such as setText() and clear().

Private access

private stops other classes from changing the field directly.

TextBox example

The text field is private; public methods provide a controlled way to use it.

```
class TextBox {
    private String text = "";

    public void setText(String value) {
        text = value;
    }

    public String getText() {
        return text;
    }

    public void clear() {
        text = "";
    }
}
```

How it protects data

The field text is private, so code outside TextBox cannot write box.text directly.

Instead, it calls methods that belong to the object:

`setText("Hello")`

`getText()`

`clear()`

These methods read or update the same object's text field.

USING THE TEXTBOX

```
TextBox box = new TextBox();
box.setText("Hello");  // store text through a method
System.out.println(box.getText());  // Hello
box.clear();  // empty the text through a method
```

In short: the class keeps the data and its methods together, and methods control access to the data.

Encapsulation: Employee Class

Private fields and methods control how employee data is read and changed

Employee keeps salary details private. Setter methods validate new values before storing them.

```
package com.salikhussain;

public class Employee {
    private int baseSalary;
    private int hourlyRate;

    public void setBaseSalary(int baseSalary) {
        if (baseSalary <= 0)
            throw new IllegalArgumentException(
                "Salary cannot be 0 or less.");
        this.baseSalary = baseSalary;
    }

    public int getBaseSalary() {
        return baseSalary;
    }

    public void setHourlyRate(int hourlyRate) {
        if (hourlyRate <= 0)
            throw new IllegalArgumentException(
                "Hourly rate cannot be 0 or negative.");
        this.hourlyRate = hourlyRate;
    }

    public int getHourlyRate() {
        return hourlyRate;
    }

    public int calculateWage(int extraHours) {
        return baseSalary + (hourlyRate * extraHours);
    }
}
```

Private fields

Main cannot directly change baseSalary or hourlyRate.

Validated setters

Setters reject zero or negative values before saving them.

Controlled methods

Getters return values; calculateWage uses the stored data.

Why this matters

Invalid values are blocked at the point where data enters the object, helping Employee keep its state valid.

Using the Employee Object

Main requests actions through methods instead of accessing fields directly

Main creates an Employee, supplies valid values through setters, and asks the object to calculate the wage.

```
package com.salikhussain;

public class Main {
    public static void main(String[] args) {
        var employee = new Employee();
        employee.setBaseSalary(50_000);
        employee.setHourlyRate(20);
        int wage = employee.calculateWage(10);

        System.out.println(wage);
    }
}
```

What happens

1. Create an Employee object.
2. Set salary to 50,000 and hourly rate to 20.
3. Calculate base salary + hourly rate x extra hours.
4. Print the calculated wage.

Wage calculation

50,000 + (20 x 10 extra hours) = 50,200

Program output: 50200

Two ways to declare the same variable

```
var employee = new Employee();
Employee employee = new Employee();
```

Here, var shortens the local declaration: Java infers Employee from new Employee(). The explicit form is equally valid and can be clearer while learning.

Encapsulation in this example

Employee owns its data and enforces its rules. Main uses public methods to request changes and calculations; it never writes the private fields directly.